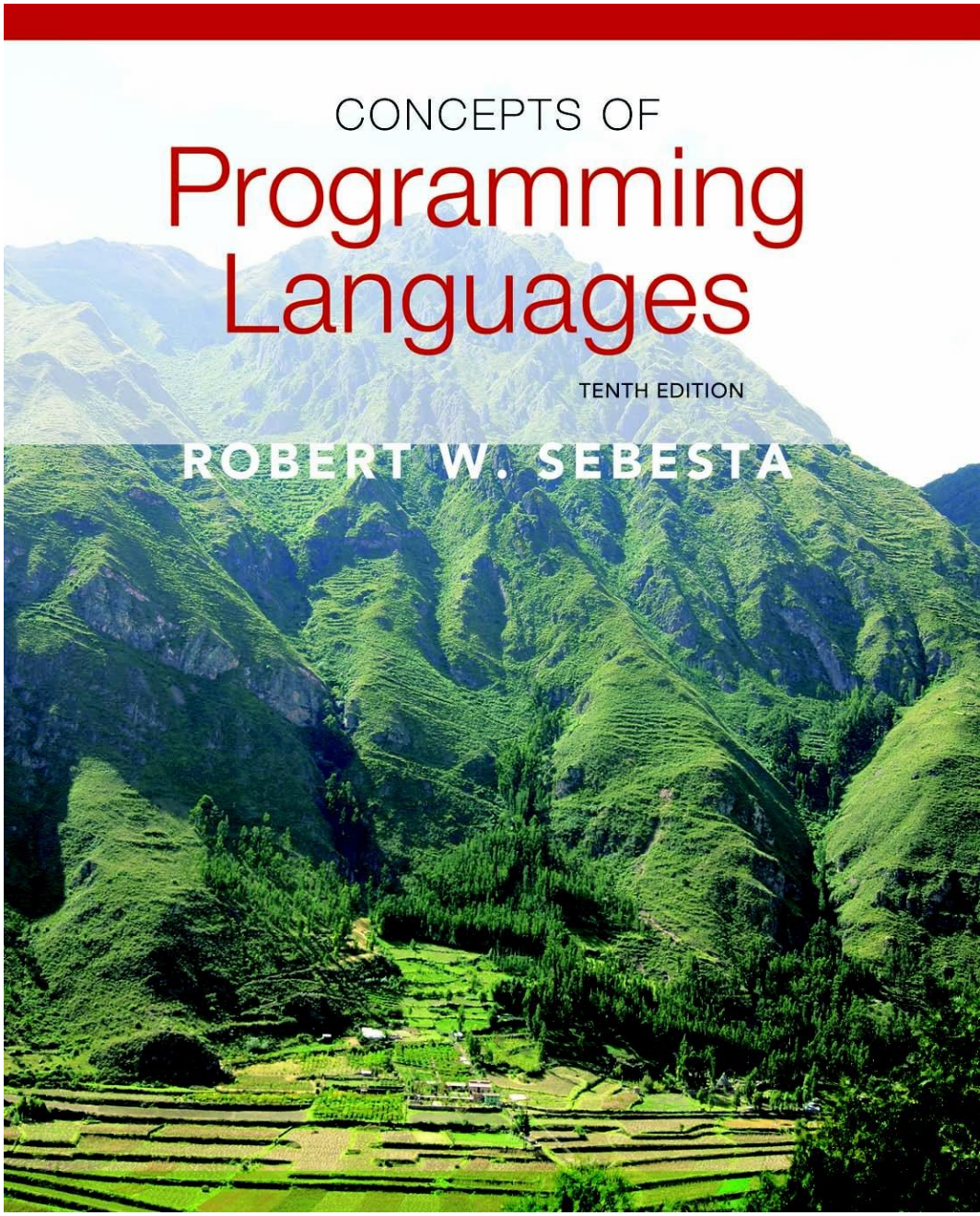
Chapter 9

Subprograms



Introduction

- Two fundamental abstraction facilities
  - Process abstraction
    - Emphasized from early days
  - Data abstraction
    - Emphasized in the1980s

Copyright © 2012 Addison-Wesley. All rights reserved.

Fundamentals of Subprograms

- Each subprogram has a single entry point
- The calling program is suspended during execution of the called subprogram
- Control always returns to the caller when the called subprogram's execution terminates

Copyright © 2012 Addison-Wesley. All rights reserved.

Basic Definitions

- A *subprogram definition* describes the interface to and the actions of the subprogram abstraction
  - In Python, function definitions are executable; in all other languages, they are non-executable
  - In Ruby, function definitions can appear either in or outside of class definitions. If outside, they are methods of `Object`. They can be called without an object, like a function
  - In Lua, all functions are anonymous
- A *subprogram call* is an explicit request that the subprogram be executed
- A *subprogram header* is the first part of the definition, including the name, the kind of subprogram, and the formal parameters
- The *parameter profile* (aka *signature*) of a subprogram is the number, order, and types of its parameters
- The *protocol* is a subprogram's parameter profile and, if it is a function, its return type

Copyright © 2012 Addison-Wesley. All rights reserved.

Basic Definitions (continued)

- Function declarations in C and C++ are often called *prototypes*
- A *subprogram declaration* provides the protocol, but not the body, of the subprogram
- A *formal parameter* is a dummy variable listed in the subprogram header and used in the subprogram
- An *actual parameter* represents a value or address used in the subprogram call statement

Copyright © 2012 Addison-Wesley. All rights reserved.

Actual/Formal Parameter Correspondence

- Positional
  - The binding of actual parameters to formal parameters is by position: the first actual parameter is bound to the first formal parameter and so forth
  - Safe and effective
- Keyword
  - The name of the formal parameter to which an actual parameter is to be bound is specified with the actual parameter
  - *Advantage*: Parameters can appear in any order, thereby avoiding parameter correspondence errors
  - *Disadvantage*: User must know the formal parameter's names

Copyright © 2012 Addison-Wesley. All rights reserved.

Formal Parameter Default Values

- In certain languages (e.g., C++, Python, Ruby, PHP), formal parameters can have default values (if no actual parameter is passed)
  - In C++, default parameters must appear last because parameters are positionally associated (no keyword parameters)

Copyright © 2012 Addison-Wesley. All rights reserved.

Variable Numbers of Parameters

- C# methods can accept a variable number of parameters as long as they are of the same type—the corresponding formal parameter is an array preceded by *params*
- In Python, the actual is a list of values and the corresponding formal parameter is a name with an asterisk
- In Lua, a variable number of parameters is represented as a formal parameter with three periods; they are accessed with a **`for`** statement or with a multiple assignment from the three periods (...)
- In Ruby, the actual parameters are sent as elements of a hash literal and the corresponding formal parameter is preceded by an asterisk.

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Procedures and Functions

- There are two categories of subprograms
  - *Procedures* are collection of statements that define parameterized computations
  - *Functions* structurally resemble procedures but are semantically modeled on mathematical functions
    - They are expected to produce no side effects
    - In practice, program functions have side effects

Copyright © 2012 Addison-Wesley. All rights reserved.

Design Issues for Subprograms

- Are local variables static or dynamic?
- Can subprogram definitions appear in other subprogram definitions?
- What parameter passing methods are provided?
- Are parameter types checked?
- If subprograms can be passed as parameters and subprograms can be nested, what is the referencing environment of a passed subprogram?
- Are functional side effects allowed?
- What types of values can be returned from functions?
- How many values can be returned from functions?
- Can subprograms be overloaded?
- Can subprogram be generic?
- If the language allows nested subprograms, are closures supported?

Copyright © 2012 Addison-Wesley. All rights reserved.

Local Referencing Environments

- Local variables can be stack-dynamic
  - Advantages
    - Support for recursion
    - Storage for locals is shared among some subprograms
  - Disadvantages
    - Allocation/de-allocation, initialization time
    - Indirect addressing
    - Subprograms cannot be history sensitive
- Local variables can be static
  - Advantages and disadvantages are the opposite of those for stack-dynamic local variables

Copyright © 2012 Addison-Wesley. All rights reserved.

Local Referencing Environments: Examples

- In most contemporary languages, locals are stack dynamic
- In C-based languages, locals are by default stack dynamic, but can be declared `static`
- The methods of C++, Java, Python, and C# only have stack dynamic locals
- In Lua, all implicitly declared variables are global; local variables are declared with `local` and are stack dynamic

Copyright © 2012 Addison-Wesley. All rights reserved.

Local Referencing Environments: Examples

```
int adder(int list[], int listlen) {
  static int sum = 0;
  int count;
  for (count = 0; count < listlen; count ++)
    sum += list [count];
  return  sum;
}
```

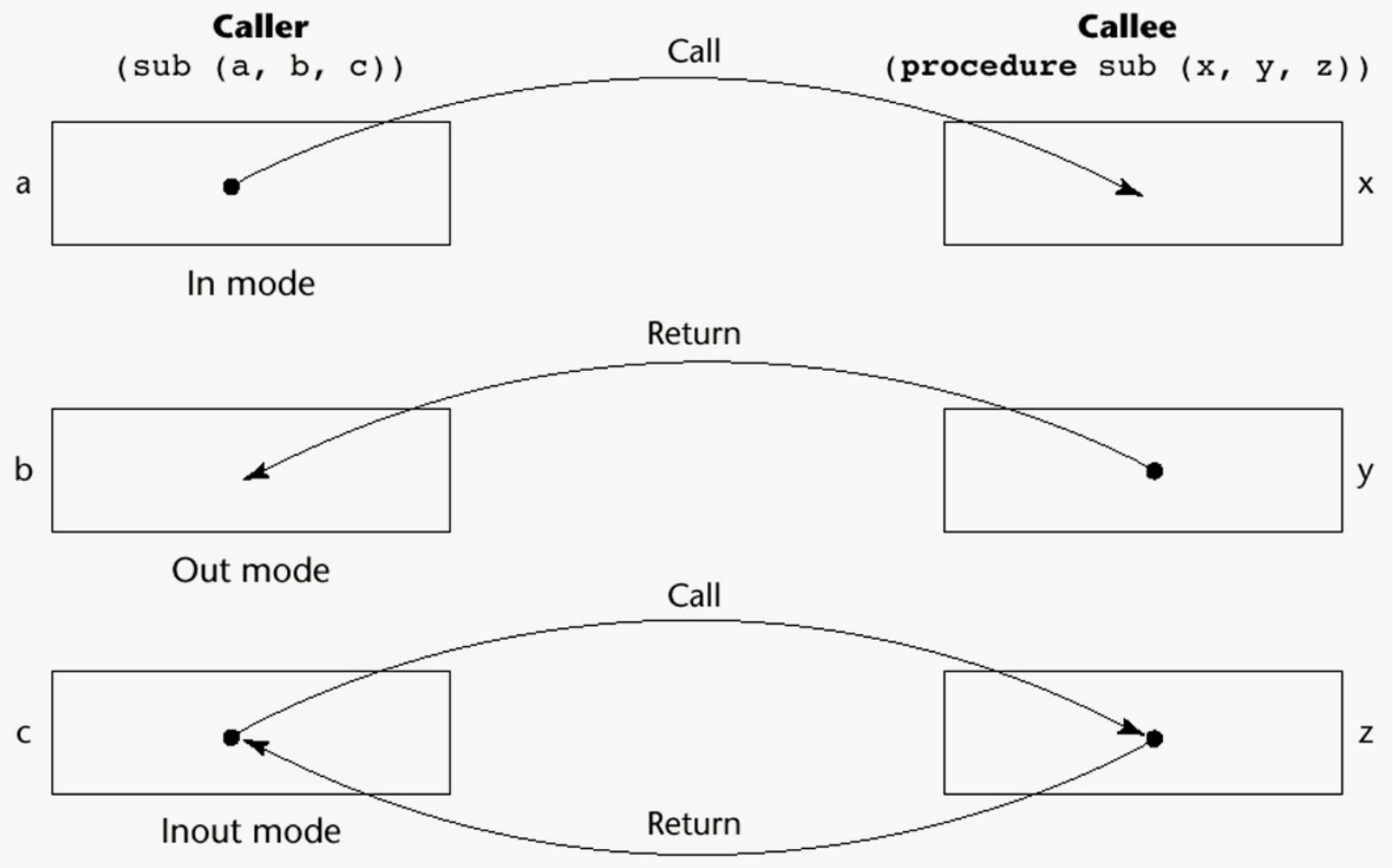
Copyright © 2012 Addison-Wesley. All rights reserved.

Semantic Models of Parameter Passing

- In mode
- Out mode
- Inout mode

Copyright © 2012 Addison-Wesley. All rights reserved.

Models of Parameter Passing

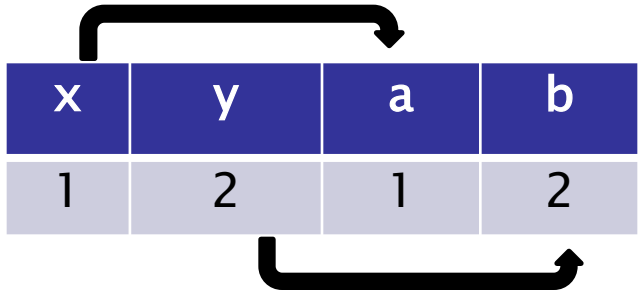


Copyright © 2012 Addison-Wesley. All rights reserved.

Semantic Models of Parameter Passing

- In mode

```
int x=1, y=2;
void f(int a, int b){
        x += a+b;
        a++;
        b = a/2;
}
int main(){
        f(x,y);
        printf("x: %d , y: %d\n",x,y);
        return 0;
}
```



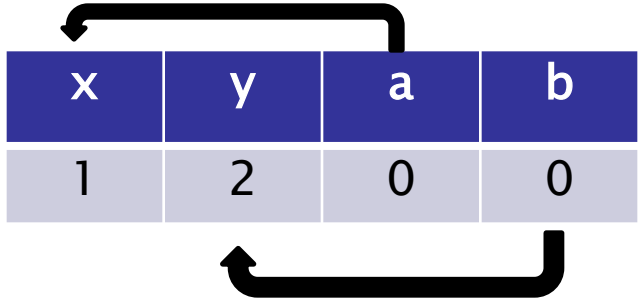
x:4 y:2

Copyright © 2012 Addison-Wesley. All rights reserved.

Semantic Models of Parameter Passing

Out mode

```
int x=1, y=2;
void f(int a, int b){
        x += a+b;
        a++;
        b = a/2;
}
int main(){
        f(x,y);
        printf("x: %d , y: %d\n",x,y);
        return 0;
}
```



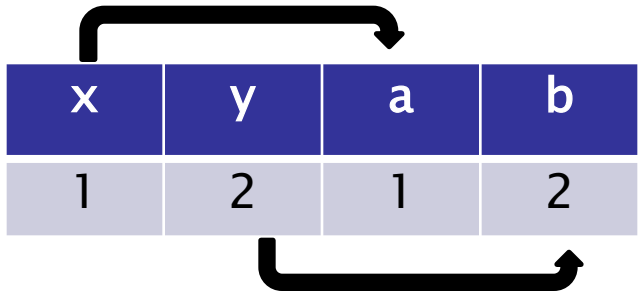
x:1 y:0

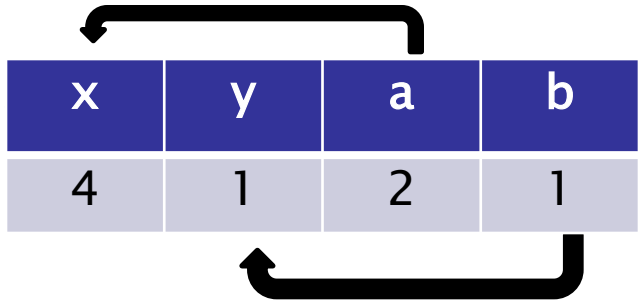
Copyright © 2012 Addison-Wesley. All rights reserved.

Semantic Models of Parameter Passing

- In-out mode

```
int x=1, y=2;
void f(int a, int b){
        x += a+b;
        a++;
        b = a/2;
}
int main(){
        f(x,y);
        printf("x: %d , y: %d\n",x
        return 0;
}
```





x:2 y:1

Copyright © 2012 Addison-Wesley. All rights reserved.

Conceptual Models of Transfer

- Physically move a value
- Move an access path to a value

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Value (In Mode)

- The value of the actual parameter is used to initialize the corresponding formal parameter
  - Normally implemented by copying
  - Can be implemented by transmitting an access path but not recommended (enforcing write protection is not easy)
  - *Disadvantages* (if by physical move): additional storage is required (stored twice) and the actual move can be costly (for large parameters)
  - *Disadvantages* (if by access path method): must write-protect in the called subprogram and accesses cost more (indirect addressing)

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Result (Out Mode)

- When a parameter is passed by result, no value is transmitted to the subprogram; the corresponding formal parameter acts as a local variable; its value is transmitted to caller's actual parameter when control is returned to the caller, by physical move
  - Require extra storage location and copy operation
- Potential problems:
  - `sub(p1, p1);` whichever formal parameter is copied back will represent the current value of `p1`
  - `sub(list[sub], sub);` Compute address of list[sub] at the beginning of the subprogram or end?

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Result (Out Mode) – C#

```
void Fixer(out int x, out int y) {
  x = 17;
  y = 35;
}
. . .
f.Fixer(out a, out a);
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Result (Out Mode) – C#

```
void DoIt(out int x, int index){
  x = 17;
  index = 42;
}
. . .
sub = 21;
f.DoIt(list[sub], sub);
```

- The address of list[sub] changes between the beginning and end of the method.
- The implementor must choose the time to bind this parameter to an address—at the time of the call or at the time of the return.
  - If the address is computed on entry to the method, the value 17 will be returned to list[21];
  - If computed just before return, 17 will be returned to list[42].

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Value-Result (Inout Mode)

- A combination of pass-by-value and pass-by-result
- Sometimes called pass-by-copy
- Formal parameters have local storage
- Disadvantages:
  - Those of pass-by-result
  - Those of pass-by-value

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Reference (Inout Mode)

- Pass an access path
- Also called pass-by-sharing
- Advantage: Passing process is efficient (no copying and no duplicated storage)
- Disadvantages
  - Slower accesses (compared to pass-by-value) to formal parameters
  - Potentials for unwanted side effects (collisions)
  - Unwanted aliases (access broadened)

```
fun(total, total);  fun(list[i], list[j]);  fun(list[i], i);
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Semantic Models of Parameter Passing

- In-out mode

```
int x=1, y=2;
void f(int a, int b){
        x += a+b;
        a++;
        b = a/2;
}
int main(){
        f(x,y);
        printf("x: %d , y: %d\n",x,y);
        return 0;
}
```

x	y
1	2

x	y
4	2

x	y
5	2

x	y
5	2

x:5 y:2

Copyright © 2012 Addison-Wesley. All rights reserved.

Pass-by-Name (Inout Mode)

- By textual substitution
- Formals are bound to an access method at the time of the call, but actual binding to a value or address takes place at the time of a reference or assignment
- Allows flexibility in late binding
- Implementation requires that the referencing environment of the caller is passed with the parameter, so the actual parameter address can be calculated

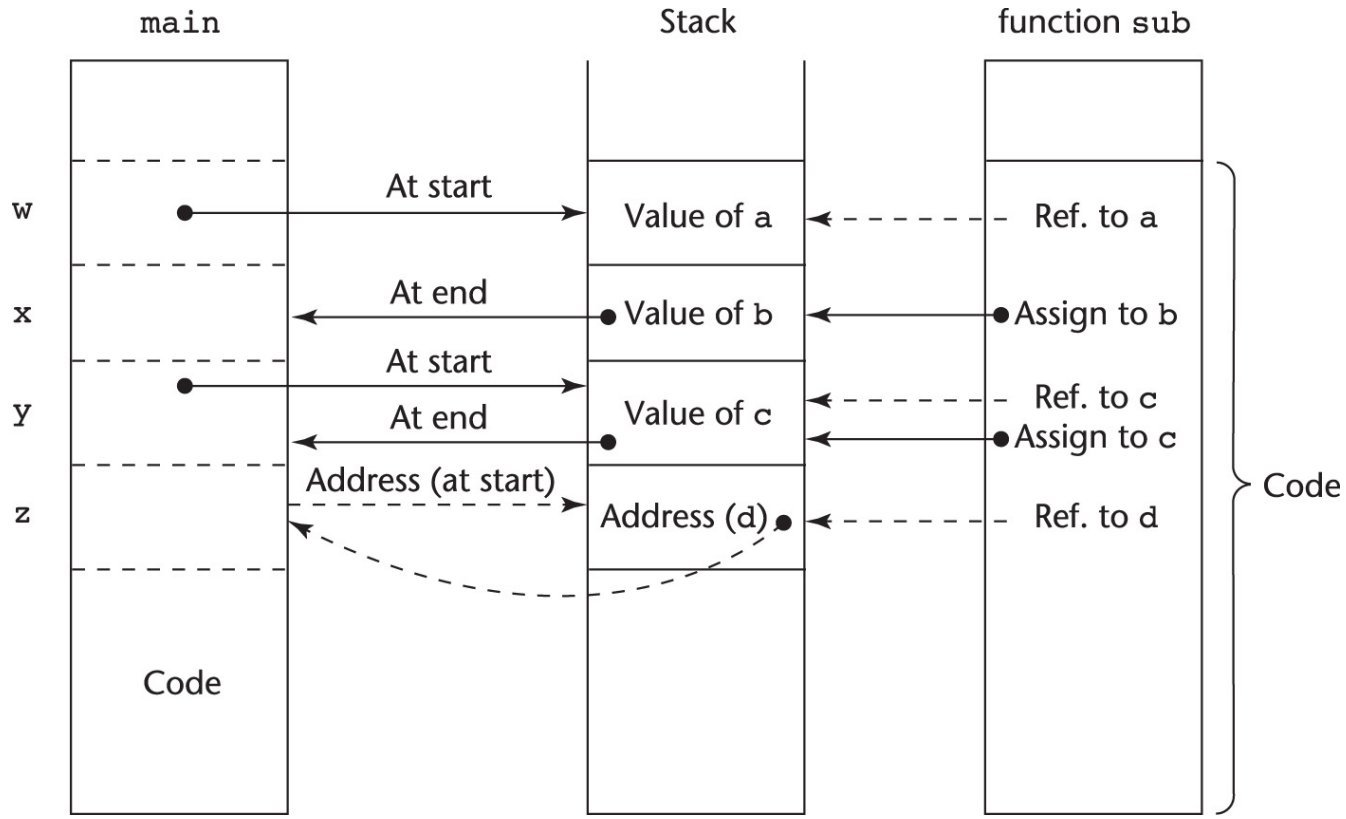
Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Implementing Parameter-Passing Methods

- In most languages parameter communication takes place through the run-time stack
- Pass-by-reference are the simplest to implement; only an address is placed in the stack

Copyright © 2012 Addison-Wesley. All rights reserved.

Implementing Parameter-Passing Methods



Function header: `void sub(int a, int b, int c, int d)`
Function call in main: `sub(w, x, y, z)`
(pass `w` by value, `x` by result, `y` by value-result, `z` by reference)

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameter Passing Methods of Major Languages

- C
  - Pass-by-value
  - Pass-by-reference is achieved by using pointers as parameters

```
void proc(int* x,int y){*x = *x+y } …
proc(&a,b);
```

- C++
  - A special pointer type called reference type for pass-by-reference

```
void proc(int &x, int y) {x = x + y }
proc(a,b);
```

- Java
  - All parameters are passed by value
  - Object parameters are passed by reference

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameter Passing Methods of Major Languages (continued)

- **C#**
  - Default method: pass-by-value
  - Pass-by-reference is specified by preceding both a formal parameter and its actual parameter with `ref`

```
void sumer(ref int oldSum, int newOne) { . . . }
. . .
sumer(ref sum, newValue);
```

- **PHP**: very similar to C#, except that either the actual or the formal parameter can specify `ref`

- **Perl**: all actual parameters are implicitly placed in a predefined array named `@_`

- **Python** and **Ruby** use pass-by-assignment (all data values are objects); the actual is assigned to the formal

Copyright © 2012 Addison-Wesley. All rights reserved.

Type Checking Parameters

- Considered very important for reliability
- **FORTRAN 77 and original C**: none
- **Pascal and Java**: it is always required
- **ANSI C and C++**: choice is made by the user
  - Prototypes
- Relatively new languages **Perl, JavaScript, and PHP** do not require type checking
- In **Python and Ruby**, variables do not have types (objects do), so parameter type checking is not possible

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Multidimensional Arrays as Parameters

- If a multidimensional array is passed to a subprogram and the subprogram is separately compiled, the compiler needs to know the declared size of that array to build the storage mapping function

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Multidimensional Arrays as Parameters: C and C++

- Programmer is required to include the declared sizes of all but the first subscript in the actual parameter

```
void fun(float *mat_ptr, int num_rows, int num_cols);
```

- **Solution**: pass a pointer to the array and the sizes of the dimensions as other parameters; the user must include the storage mapping function in terms of the size parameters

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Multidimensional Arrays as Parameters: Java and C#

- Arrays are objects; they are all single-dimensioned, but the elements can be arrays
- Each array inherits a named constant (`length` in Java, `Length` in C#) that is set to the length of the array when the array object is created

```
float sumer(float mat[][]) {
   float sum = 0.0f;
   for (int row = 0; row < mat.length; row++) {
     for (int col = 0; col < mat[row].length; col++) {
       sum += mat[row][col];
     }  //** for (int row . . .
   } //** for (int col . . .
   return sum;
}
```

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Design Considerations for Parameter Passing

- Two important considerations
  - Efficiency
  - One-way or two-way data transfer
- But the above considerations are in conflict
  - Good programming suggest limited access to variables, which means one-way whenever possible
  - But pass-by-reference is more efficient to pass structures of significant size

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Parameters that are Subprogram Names

- It is sometimes convenient to pass subprogram names as parameters
- Issues:
  1. Are parameter types checked?
  2. What is the correct referencing environment for a subprogram that was sent as a parameter?

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Parameters that are Subprogram Names: Referencing Environment

- *Shallow binding*: The environment of the call statement that enacts the passed subprogram
  - Appropriate for dynamic-scoped languages
- *Deep binding*: The environment of the definition of the passed subprogram
  - Most natural for static-scoped languages
- *Ad hoc binding*: The environment of the call statement that passed the subprogram

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Parameters that are Subprogram Names: Referencing Environment

```
function sub1() {
    var x;
    function sub2() {
        alert(x);
    } // sub2
    function sub3(){
        var x;
        x = 3;
        sub4(sub2);
    } // sub3
    function sub4(subx) {
        var x;
        x = 4;
        subx();
    } // sub4
    x = 1;
    sub3();
} // sub1
```

Passed subprogram sub2

Output:

shallow binding: 4
deep binding: 1
ad-hoc binding: 3

Copyright © 2017 Pearson Education, Ltd. All rights reserved.